# Supplementary Material to: Who Pays for the Bit? Evidence-Aware Energy Attribution Across Shared 5G and 6G Network Infrastructure

Ghufran Shakeel

*24 October 2026*

## S1. Literature matrix and research gap

**Gap.** Public sources show that attributing energy to a slice or service is already a requirement and a work item. We found no source, in the parts we could read, that also rejects overlapping measurement boundaries, keeps unallocated energy as a reported residual, states the range a service's energy can take without a declared baseline policy, and states when per-service energy cannot be identified from meter totals and withholds the figure. The paper's claim is limited to that combination and remains a claim until full texts are read.

"Status" says how much of each source we read. "Snippet" means a search extract or quoted draft, not the full document.

| # | Source | What it establishes | Limit for our purpose | Status |
|---|---|---|---|---|
| 1 | ETSI TS 128 310 V17.8.0 (3GPP TS 28.310 Release 17) | Finest measurable grain in a virtualized host is the node; function energy estimated as a vCPU-usage proportion of node energy; RAN counts DU energy only; operator data centres are telecom sites | Release 17 only; no residual or uncertainty in the estimate | Clauses 4.1, 6.1.2, 6.3.2 read |
| 2 | ITU-R M.2160-0 (11/2023) | Sustainability is a foundational aspiration of IMT-2030; energy efficiency defined in bit/Joule | Framework document, no attribution method | Clauses 2.1, 2.2.9, 4 read |
| 3 | ISO/IEC 30134-9:2022 | Data-centre and IT energy boundaries; twelve-month data; accuracy categories and derivatives for the water KPI | Water KPI, not telecom energy attribution | First five pages of preview read |
| 4 | O-RAN ALLIANCE announcement, 71 documents since Nov. 2025 | New network energy-saving use cases in several working groups | Energy saving, not attribution; specifications not opened | Announcement read |
| 5 | 3GPP TS 28.554, clause 6.7.3.3 | Slice energy is the sum of the energy of its network functions; shared gNB and UPF attributed by data-volume share | Other shared core functions not addressed; shared function instances out of scope | Clause text via mirror |
| 6 | 3GPP SA5 draft pCR to TR 28.880, S5-247121 | Proposes per-slice energy for shared gNBs and 5GC functions | Draft working document | Snippet |
| 7 | 3GPP TS 22.261, clause 6.15a.4.2 | Per-slice and per-subscriber monitoring; attribution as accurate as possible | A requirement, not a method | Clause text via mirror |
| 8 | 3GPP Rel-19 CR 28.201, S5-244012rev1 | Adds estimated slice energy to charging information | Uses the TS 28.554 KPI as input | Snippet |
| 9 | US Patent 11,711,771 B2 | Slice energy as the difference with and without the slice | Needs both states to be runnable | Abstract and claim snippet |
| 10 | TNO, EXIGENCE poster, Zenodo, 2 June 2026 | Power measurement and attribution at network- | Poster description | Abstract only |

| # | Source | What it establishes | Limit for our purpose | Status |
|---|---|---|---|---|
| | | function and user level | | |
| 11 | Hossfeld et al., IEEE Access 12, 2024 | Limits of apportioning network energy to services by energy intensity | Not an accounting procedure | Abstract only |
| 12 | Deruyck et al., EURASIP JWCN 2012:248, Table 1 | Macro base station: 605 W constant, 1,067.7 W load-dependent | Older equipment generation | Table read |
| 13 | Barroso and Hölzle, IEEE Computer 40(12), 2007 | Servers draw a large share of peak power at low utilization | The 50% at 10% utilization figure is from a secondary summary; modern servers are more proportional | Abstract and summary |
| 14 | JCGM 101:2008 | Monte Carlo propagation of distributions | Not applied to allocation policy | Title and DOI checked |

Cited from background knowledge and not opened in this work: CR 0090, the full texts of 3GPP TR 26.942 and ETSI ES 203 228, the individual O-RAN specifications, ISO/IEC 30134-2, JCGM 100, Shapley (1953), and the full text of the IEEE Access 2024 paper.

**Verification log (24 October 2026).** Checked against primary or mirrored sources: the author list and DOI of the EURASIP 2012 paper (Deruyck, Tanghe, Joseph, Martens); the author list of the IEEE Access 2024 paper (Hossfeld, Wunderer, Loh, Schien); the title of TR 26.942 (Study on Media Energy Consumption Exposure and Evaluation Framework, Release 19); ETSI ES 203 228 V1.4.1 (2022-04); the text of TS 28.554 clause 6.7.3.3 and TS 22.261 clause 6.15a.4.2 through the iTecSpec mirror; and the JCGM 101:2008 title and DOI on the BIPM site. Read from uploaded documents: ITU-R M.2160-0 (clauses 2.1, 2.2.9 and 4); ETSI TS 128 310 V17.8.0 (clauses 4.1, 6.1.2 and 6.3.2); ISO/IEC 30134-9:2022 (first five pages of the iTeh preview); and the O-RAN ALLIANCE announcement of 71 new or updated documents. The uploaded ISO document is Part 9 (water usage effectiveness), not Part 2 (power usage effectiveness), so the paper cites Part 9, which restates the Part 2 definitions of total data-centre and IT equipment energy. The Release 19 change request on energy efficiency as a service criterion is not in the Release 17 text and remains unread. Entries for Holtkamp (2013) and Computer Communications 50 (2014) were removed because the paper did not use them. The patent record showed no assignee, so none is given. Still unopened: TS 28.310 Releases 18 and 19, CR 0090, ISO/IEC 30134-2, JCGM 100:2008, Shapley (1953), the O-RAN specifications behind the announcement, and the full texts of TR 26.942 and ES 203 228.

## S2. Full scenario table

All values are generated by the code in Section S3 (seed 20260101) and stored in results.json. Errors are the share of energy assigned to the wrong service, 0.5 Σ |estimate − reference| / total. EAEA is scored on the services and domains it reports; A to D are shares of trials; Spread is the mean range of a service's end-to-end share across three baseline policies. Each row is 2,000 trials.

**Table S1. All 36 scenarios of the first study.**

| Traffic | β | P(miss) | σ_w | B1 | B2 | EAEA | A | B | C | D | Spread |
|---|---|---|---|---|---|---|---|---|---|---|---|
| uniform | 0.2 | 0.0 | 0.05 | 0.077 | 0.056 | 0.025 | 1.000 | 0.000 | 0.000 | 0.000 | 0.014 |
| uniform | 0.2 | 0.0 | 0.2 | 0.077 | 0.090 | 0.063 | 1.000 | 0.000 | 0.000 | 0.000 | 0.015 |
| uniform | 0.2 | 0.1 | 0.05 | 0.078 | 0.058 | 0.026 | 0.111 | 0.343 | 0.518 | 0.029 | 0.015 |
| uniform | 0.2 | 0.1 | 0.2 | 0.078 | 0.088 | 0.063 | 0.128 | 0.334 | 0.512 | 0.026 | 0.015 |
| uniform | 0.2 | 0.3 | 0.05 | 0.076 | 0.062 | 0.026 | 0.003 | 0.017 | 0.501 | 0.480 | 0.014 |
| uniform | 0.2 | 0.3 | 0.2 | 0.078 | 0.084 | 0.062 | 0.002 | 0.014 | 0.514 | 0.470 | 0.015 |
| uniform | 0.5 | 0.0 | 0.05 | 0.138 | 0.129 | 0.027 | 1.000 | 0.000 | 0.000 | 0.000 | 0.036 |
| uniform | 0.5 | 0.0 | 0.2 | 0.136 | 0.144 | 0.056 | 1.000 | 0.000 | 0.000 | 0.000 | 0.037 |
| uniform | 0.5 | 0.1 | 0.05 | 0.136 | 0.128 | 0.027 | 0.117 | 0.343 | 0.505 | 0.034 | 0.036 |
| uniform | 0.5 | 0.1 | 0.2 | 0.137 | 0.143 | 0.056 | 0.116 | 0.344 | 0.517 | 0.022 | 0.037 |
| uniform | 0.5 | 0.3 | 0.05 | 0.137 | 0.129 | 0.027 | 0.001 | 0.011 | 0.487 | 0.500 | 0.036 |
| uniform | 0.5 | 0.3 | 0.2 | 0.137 | 0.141 | 0.055 | 0.002 | 0.019 | 0.522 | 0.458 | 0.037 |

| Traffic | β | P(miss) | σ_w | B1 | B2 | EAEA | A | B | C | D | Spread |
|---|---|---|---|---|---|---|---|---|---|---|---|
| uniform | 0.8 | 0.0 | 0.05 | 0.208 | 0.202 | 0.031 | 1.000 | 0.000 | 0.000 | 0.000 | 0.058 |
| uniform | 0.8 | 0.0 | 0.2 | 0.206 | 0.213 | 0.064 | 1.000 | 0.000 | 0.000 | 0.000 | 0.059 |
| uniform | 0.8 | 0.1 | 0.05 | 0.206 | 0.202 | 0.031 | 0.134 | 0.326 | 0.510 | 0.030 | 0.058 |
| uniform | 0.8 | 0.1 | 0.2 | 0.206 | 0.212 | 0.063 | 0.131 | 0.322 | 0.519 | 0.029 | 0.059 |
| uniform | 0.8 | 0.3 | 0.05 | 0.207 | 0.203 | 0.031 | 0.001 | 0.010 | 0.508 | 0.481 | 0.058 |
| uniform | 0.8 | 0.3 | 0.2 | 0.207 | 0.210 | 0.064 | 0.002 | 0.015 | 0.517 | 0.466 | 0.059 |
| skewed | 0.2 | 0.0 | 0.05 | 0.325 | 0.097 | 0.026 | 1.000 | 0.000 | 0.000 | 0.000 | 0.046 |
| skewed | 0.2 | 0.0 | 0.2 | 0.326 | 0.115 | 0.055 | 1.000 | 0.000 | 0.000 | 0.000 | 0.046 |
| skewed | 0.2 | 0.1 | 0.05 | 0.328 | 0.122 | 0.032 | 0.127 | 0.327 | 0.512 | 0.034 | 0.046 |
| skewed | 0.2 | 0.1 | 0.2 | 0.327 | 0.138 | 0.060 | 0.127 | 0.312 | 0.531 | 0.030 | 0.047 |
| skewed | 0.2 | 0.3 | 0.05 | 0.329 | 0.167 | 0.026 | 0.000 | 0.013 | 0.532 | 0.455 | 0.046 |
| skewed | 0.2 | 0.3 | 0.2 | 0.326 | 0.181 | 0.056 | 0.002 | 0.016 | 0.509 | 0.473 | 0.047 |
| skewed | 0.5 | 0.0 | 0.05 | 0.400 | 0.237 | 0.031 | 1.000 | 0.000 | 0.000 | 0.000 | 0.116 |
| skewed | 0.5 | 0.0 | 0.2 | 0.398 | 0.244 | 0.057 | 1.000 | 0.000 | 0.000 | 0.000 | 0.117 |
| skewed | 0.5 | 0.1 | 0.05 | 0.394 | 0.249 | 0.034 | 0.130 | 0.332 | 0.514 | 0.025 | 0.114 |
| skewed | 0.5 | 0.1 | 0.2 | 0.400 | 0.258 | 0.061 | 0.127 | 0.331 | 0.510 | 0.033 | 0.118 |
| skewed | 0.5 | 0.3 | 0.05 | 0.397 | 0.279 | 0.032 | 0.002 | 0.015 | 0.504 | 0.479 | 0.116 |
| skewed | 0.5 | 0.3 | 0.2 | 0.394 | 0.283 | 0.056 | 0.001 | 0.011 | 0.500 | 0.487 | 0.115 |
| skewed | 0.8 | 0.0 | 0.05 | 0.479 | 0.378 | 0.041 | 1.000 | 0.000 | 0.000 | 0.000 | 0.184 |
| skewed | 0.8 | 0.0 | 0.2 | 0.481 | 0.384 | 0.071 | 1.000 | 0.000 | 0.000 | 0.000 | 0.186 |
| skewed | 0.8 | 0.1 | 0.05 | 0.479 | 0.384 | 0.042 | 0.119 | 0.344 | 0.505 | 0.032 | 0.184 |
| skewed | 0.8 | 0.1 | 0.2 | 0.480 | 0.390 | 0.072 | 0.117 | 0.323 | 0.528 | 0.032 | 0.186 |
| skewed | 0.8 | 0.3 | 0.05 | 0.480 | 0.403 | 0.040 | 0.001 | 0.016 | 0.512 | 0.471 | 0.185 |
| skewed | 0.8 | 0.3 | 0.2 | 0.482 | 0.405 | 0.070 | 0.001 | 0.016 | 0.503 | 0.480 | 0.187 |

## S3. Code

Python 3 with numpy only. The same files are in the repository. Save the first listing as eaea_experiment.py and the second as case_study.py. The first writes results.json to the working directory.

### D.1 eaea_experiment.py

```
"""EAEA reference implementation + synthetic evaluation. Seed-fixed;
numpy only.
Run: python eaea_experiment.py  -> prints tables, writes results.json"""
import numpy as np, json, itertools
S, D = 5, 4                       # services, domains (RAN, transport,
core, edge)
MIX = np.array([0.8, 0.9, 0.5, 0.1])   # weight of traffic in true
variable-energy driver
SIG_M, EPS_B = 0.05, 0.10         # meter error (total), baseline-model
error
N = 2000

def dirich(rng, a, shape): return rng.dirichlet(np.full(S, a),
size=shape)

def trial_batch(rng, alpha, beta, pmiss, sigw, n):
    t = dirich(rng, alpha, n)                       # n,S traffic shares
    c = dirich(rng, alpha, (n, D))                  # n,D,S
compute/resource shares
    drv = MIX[None, :, None]*t[:, None, :] + (1-MIX[None, :, None])*c   #
true var driver n,D,S
    cap = dirich(rng, 2.0, (n, D))                  # true reserved-
```

```
capacity share (reference baseline rule)
    Ed = 1000*np.array([3, 1, 1.5, 2.5])[None, :]*np.ones((n, D))
    B = np.broadcast_to(np.asarray(beta, float), (n, D))
    ref = (1-B)[..., None]*Ed[..., None]*drv + B[..., None]*Ed[...,
None]*cap            # n,D,S reference
    Em = Ed*(1+SIG_M*rng.standard_normal((n, D)))
# measured totals
    nz = lambda x: np.clip(x*(1+sigw*rng.standard_normal(x.shape)), 1e-3,
None)
    w = nz(drv); w /= w.sum(-1, keepdims=True)
    miss = rng.random((n, D, S)) < pmiss
    tt = np.broadcast_to(t[:, None, :], (n, D, S))
    # Baseline 1: traffic-proportional on whole measured total
    b1 = Em[..., None]*tt
    # Baseline 2: resource-weighted, silent fallback to traffic where
missing
    w2 = np.where(miss, tt, w); w2 = w2/w2.sum(-1, keepdims=True)
    b2 = Em[..., None]*w2
    # EAEA: baseline split by noisy declared model, var by weights, base
by capacity proxy
    bh = np.clip(B*(1+EPS_B*rng.standard_normal((n, D))), 0, 1)
    capo = nz(cap); capo /= capo.sum(-1, keepdims=True)
    dom_incomplete = miss.any(-1)                  # n,D
    k = dom_incomplete.sum(-1)                     # n
    e3 = Em[..., None]*((1-bh)[..., None]*w2 + bh[..., None]*capo)  #
fallback flagged via category
    # conservation check
    resid = np.abs(e3.sum(-1) - Em).max()
    cat = np.where((k == 0) & (SIG_M <= 0.05), 0, np.where(k <= 1, 1,
np.where(k <= 3, 2, 3)))  # A,B,C,D
    # errors: 0.5*sum|est-ref| / true total (in-scope domains)
    def err(est, mask):
        num = (0.5*np.abs(est-ref).sum(-1)*mask).sum(-1)
        den = (Ed*mask).sum(-1)
        return np.where(den > 0, num/np.maximum(den, 1e-9), np.nan)
    full = np.ones((n, D))
    scope = np.where((cat == 2)[:, None],
(~dom_incomplete).astype(float), full)
    scope = np.where((cat == 3)[:, None], 0.0, scope)
    # policy sensitivity: spread of service-total share across 3 baseline
policies
    pol = [tt, capo, np.full((n, D, S), 1/S)]
    tot = [ (((1-bh)[..., None]*w2 + bh[..., None]*p)*Ed[...,
None]).sum(1)/Ed.sum(1, keepdims=True) for p in pol]
    spread = (np.max(tot, 0)-np.min(tot, 0)).mean(-1)
    return dict(e1=err(b1, full), e2=err(b2, full), e3=err(e3, scope),
e1s=err(b1, scope), e2s=err(b2, scope),
                cat=cat, resid=resid, spread=spread,
                unalloc=0.0)

def summarize(x): x = x[~np.isnan(x)]; return float(x.mean()),
float(1.96*x.std(ddof=1)/np.sqrt(len(x))), int(len(x))
```

```
rng = np.random.default_rng(20260101)
rows = []; pooled = {}
for (an, alpha), beta, pm, sw in itertools.product([("uniform", 20.0),
("skewed", 0.3)], [0.2, 0.5, 0.8], [0.0, 0.1, 0.3], [0.05, 0.20]):
    r = trial_batch(rng, alpha, beta, pm, sw, N)
    rows.append(dict(traffic=an, beta=beta, pmiss=pm, sigw=sw,
                     e1=r["e1"].mean(), e2=r["e2"].mean(),
                     e3_reported=np.nanmean(r["e3"]),
                     e2_same_scope=np.nanmean(r["e2s"]),
                     p=[float((r["cat"] == i).mean()) for i in range(4)],
                     resid=float(r["resid"]),
spread=float(r["spread"].mean())))
    for kx in ("e1", "e2", "e3", "e2s", "cat", "spread"):
pooled.setdefault(kx, []).append(r[kx])
    pooled.setdefault("meta", []).append((an, beta, pm, sw))
P = {k: np.concatenate(v) for k, v in pooled.items() if k != "meta"}
def grp(key_fn):
    out = {}
    for rw in rows: out.setdefault(key_fn(rw), []).append(rw)
    return {k: dict(e1=np.mean([r["e1"] for r in v]), e2=np.mean([r["e2"]
for r in v]),
                    e2s=np.mean([r["e2_same_scope"] for r in v]),
e3=np.nanmean([r["e3_reported"] for r in v]),
                    pA=np.mean([r["p"][0] for r in v]),
pB=np.mean([r["p"][1] for r in v]),
                    pC=np.mean([r["p"][2] for r in v]),
pD=np.mean([r["p"][3] for r in v]),
                    spread=np.mean([r["spread"] for r in v])) for k, v in
sorted(v_items(out))}
def v_items(o): return o.items()
res = {}
res["scenario_grid"] = rows
res["by_pmiss"] = grp(lambda r: r["pmiss"]); res["by_beta"] = grp(lambda
r: r["beta"])
res["by_traffic"] = grp(lambda r: r["traffic"]); res["by_sigw"] =
grp(lambda r: r["sigw"])
res["overall"] = {k: summarize(P[k]) for k in ("e1", "e2", "e3")}
res["by_category"] = {}
for i, nm in enumerate("ABCD"):
    m = P["cat"] == i
    res["by_category"][nm] = dict(share=float(m.mean()),
        eaea=summarize(P["e3"][m]) if i < 3 and m.any() else None,
        b2_same_scope=summarize(P["e2s"][m]) if i < 3 and m.any() else
None,
        b2_full=summarize(P["e2"][m]) if m.any() else None)
res["max_residual_J"] = max(r["resid"] for r in rows)
res["n_trials"] = N*len(rows); res["n_scenarios"] = len(rows)
# controlled overlap test
rack, server = 100.0, 40.0
res["overlap_test"] = dict(naive_sum=rack+server, true=rack,
eaea_sum=rack+server-server)
```

```
# B1 vs B2 paired win fraction (EAEA vs B2 same scope, reported trials
A/B/C)
m = (P["cat"] < 3) & ~np.isnan(P["e3"])
res["paired_win_vs_b2_same_scope"] = float((P["e3"][m] < P["e2s"]
[m]).mean())
json.dump(res, open("results.json", "w"), indent=1, default=float)

# ---- Literature-calibrated baseline fractions (still synthetic
traffic/measurement noise) ----
# RAN: macro BS P = 605 W constant + 1067.7 W * load (Deruyck et al.,
EURASIP JWCN 2012:248, Table 1); load ~ U(0.1, 0.6)
# Core/Edge hosts: idle = 50% of peak power, linear in utilisation
(Barroso & Hoelzle 2007); utilisation ~ U(0.1, 0.5)
# Transport: ASSUMED beta ~ U(0.5, 0.9) (no verified source) -- flagged
as assumption in the paper
class CalibRng:
    pass
def calib_beta(rng, n):
    rho = rng.uniform(0.1, 0.6, n); ran = 605/(605+1067.7*rho)
    u = rng.uniform(0.1, 0.5, (n, 2)); srv = 0.5/(0.5+0.5*u)
    tr = rng.uniform(0.5, 0.9, n)
    return np.column_stack([ran, tr, srv[:, 0], srv[:, 1]])
crows = []
for (an, alpha), pm, sw in itertools.product([("uniform", 20.0),
("skewed", 0.3)], [0.0, 0.1, 0.3], [0.05, 0.20]):
    bt = calib_beta(rng, N)
    r = trial_batch(rng, alpha, bt, pm, sw, N)
    m = ~np.isnan(r["e3"])
    crows.append(dict(traffic=an, pmiss=pm, sigw=sw, e1=r["e1"].mean(),
e2=r["e2"].mean(),
                      e3=np.nanmean(r["e3"]), e2s=np.nanmean(r["e2s"]),
spread=float(r["spread"].mean()),
                      cat=r["cat"], e3v=r["e3"], e2sv=r["e2s"],
e1v=r["e1"], e2v=r["e2"], bt=bt.mean(0).tolist()))
cat_all = np.concatenate([c["cat"] for c in crows]); e3_all =
np.concatenate([c["e3v"] for c in crows])
e2s_all = np.concatenate([c["e2sv"] for c in crows]); e1_all =
np.concatenate([c["e1v"] for c in crows]); e2_all =
np.concatenate([c["e2v"] for c in crows])
cal = dict(n_trials=N*len(crows), mean_beta_by_domain=np.mean([c["bt"]
for c in crows], 0).tolist(),
           overall=dict(e1=summarize(e1_all), e2=summarize(e2_all),
e3=summarize(e3_all)),
           cat_share={k: float((cat_all == i).mean()) for i, k in
enumerate("ABCD")},
           by_cat={k: dict(eaea=summarize(e3_all[cat_all == i]),
b2=summarize(e2s_all[cat_all == i])) for i, k in enumerate("ABC")},
           by_traffic={t: dict(e1=float(np.mean([c["e1"] for c in crows
if c["traffic"] == t])), e2=float(np.mean([c["e2"] for c in crows if
c["traffic"] == t])), e3=float(np.mean([c["e3"] for c in crows if
c["traffic"] == t])), spread=float(np.mean([c["spread"] for c in crows if
c["traffic"] == t]))) for t in ("uniform", "skewed")},
```

```
            win=float((e3_all[cat_all < 3] < e2s_all[cat_all <
3]).mean()))
res["calibrated"] = cal
json.dump(res, open("results.json", "w"), indent=1, default=float)
print("CALIBRATED", json.dumps(cal, indent=1, default=float))

def show(t, d):
    print("==", t)
    for k, v in d.items(): print(k, {a: round(b, 3) for a, b in
v.items()})
for t in ("by_pmiss", "by_beta", "by_traffic", "by_sigw"): show(t,
res[t])
print(json.dumps({k: res[k] for k in ("overall", "by_category",
"max_residual_J", "n_trials", "n_scenarios", "overlap_test",
"paired_win_vs_b2_same_scope")}, indent=1, default=float))
```

**D.2 case_study.py**

```
"""Deterministic case study. Equipment parameters: published macro-BS
model (605 W constant, 1067.7 W load-dependent at full load; EURASIP JWCN
2012:248, Table 1).
Service traffic shares are ILLUSTRATIVE ASSUMPTIONS (no public per-
service ground truth exists). No randomness."""
import numpy as np
P0, PL = 605.0, 1067.7
share = {"eMBB": 0.70, "mMTC": 0.20, "URLLC": 0.10}
resv  = {"eMBB": 0.50, "mMTC": 0.20, "URLLC": 0.30}   # assumed reserved-
capacity shares (URLLC reserves headroom)
for rho in (0.1, 0.3, 0.6):
    P = P0 + PL*rho; var = PL*rho; beta = P0/P
    print(f"\nload={rho}: P={P:.1f} W, baseline share={beta:.1%},
energy/h={P*3600/3.6e6:.3f} kWh")
    for s in share:
        v = var*share[s]
        pol_traffic = P*share[s]
        pol_equal   = v + P0/3
        pol_resv    = v + P0*resv[s]
        lo, hi = v, v + P0
        print(f"  {s:6s} traffic={pol_traffic:6.1f}  equal-
base={pol_equal:6.1f}  reserved-base={pol_resv:6.1f}  bounds=[{lo:6.1f},
{hi:6.1f}] W  max/min policy
ratio={max(pol_traffic,pol_equal,pol_resv)/min(pol_traffic,pol_equal,pol_
resv):.2f}")
# identifiability: meter-only regression E_t = b + sum_k a_k x_kt
t = np.arange(24);
x1 = 0.5+0.4*np.sin(2*np.pi*t/24); x2 = 0.2+0.1*np.cos(2*np.pi*t/24+1.0);
x3 = 0.5*x1       # x3 exactly proportional to x1
for name, X in (("independent loads", np.column_stack([np.ones(24), x1,
x2, 0.1+0.05*np.sin(2*np.pi*t/8)])),
                ("service 3 proportional to service 1",
np.column_stack([np.ones(24), x1, x2, x3]))):
    print(f"{name}: rank={np.linalg.matrix_rank(X)} of {X.shape[1]}
columns, cond={np.linalg.cond(X):.3g}")
```